

Die Lösungen für diese Übung sind abzugeben bis Sonntag, den 24.05.2026 um 23:59h.

Deferred Rendering

Praktischer Aufgabenteil (5 Punkte)

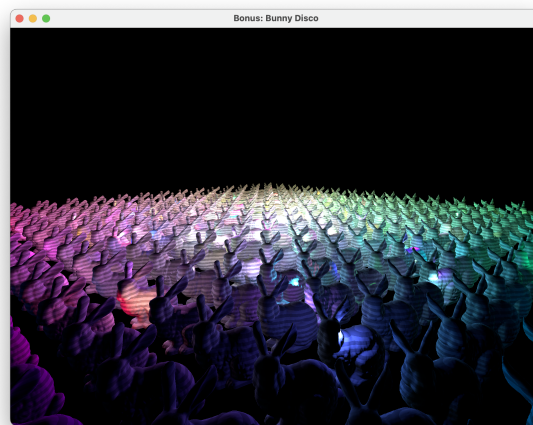
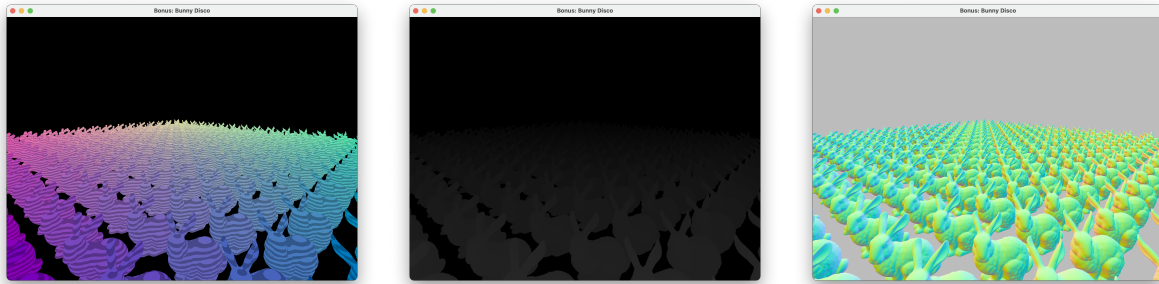


Abbildung 1: So soll das Ergebnis aussehen: Viele Hasen, viele Lichter.

In dieser Aufgabe wollen wir eine Hasenparty für die Stanford-Hasen veranstalten, mit vielen Hasen und vielen bunten Lichtern. Dabei stoßen wir jedoch schnell auf ein Problem: Angenommen wir haben n Hasen und m Lichtquellen. Unser bisheriger Renderer würde jedes Objekt in einem einzelnen Draw-Call auf den Bildschirm zeichnen und für jedes Fragment jeweils über alle Lichtquellen iterieren und die notwendigen Beleuchtungsberechnungen machen, auch wenn Teile des Objekts gar nicht sichtbar sind, weil diese in einem folgenden Draw-Call von anderen Fragmenten überdeckt werden. Im schlimmsten Fall müssen wir so bis zu $m \times n$ Beleuchtungsberechnungen *pro Pixel* ausführen, was bei vielen Objekten und/oder Lichtern schnell teuer werden kann. Diesen Rendering-Ansatz nennt man *Forward-Rendering*.

In diesem Bonusblatt werden wir einen alternativen Ansatz kennenlernen, der in vielen Echtzeit-Graphikengines Anwendung findet: *Deferred-Rendering*. Beim Deferred-Rendering werden die Beleuchtungsberechnungen verzögert (daher “deferred”), indem in einem ersten Rasterisierungs-Schritt nur abstrakte Szenen-Informationen wie Farbwerte, Normalen, Tiefeninformationen und Materialeigenschaften gerendert werden:



In einem zweiten Schritt werden diese Szenen-Informationen dann benutzt um nun die Beleuchtungs-berechnungen für jeden Pixel auszuführen. Der große Vorteil ist, dass wir nun für jeden Pixel nur noch m Beleuchtungsberechnungen anstellen müssen und uns unter Umständen viel Rechenarbeit sparen können. Schwierig wird es nur mit Transparenz, doch die betrachten wir hier noch nicht.

Schauen wir jetzt in den Code:

Implementation

Um unsere Beleuchtungsberechnungen aufschieben zu können, müssen wir die Szeneninformationen irgendwo zwischenspeichern. Das machen wir in einem *Framebuffer*, ein Framebuffer ist eine Sammlung von Texturen, in die wir direkt zeichnen können. Wir haben Framebuffer schon benutzt ohne es zu wissen, denn auch um auf den Bildschirm zu zeichnen brauchen wir einen Framebuffer, den unser Fenster automatisch erzeugt. Dieser Framebuffer ist der Default-Framebuffer mit dem Index 0. Wir brauchen jedoch einen zweiten Framebuffer, den *gBuffer* (Geometry-Buffer), den wir genauso anlegen wie alles andere in OpenGL und zwar mit den `glCreateFramebuffers` und `glDeleteFramebuffers` Befehlen. Wie bei den anderen OpenGL-Objekten ist die Erzeugung und Zerstörung in einer Klasse *Framebuffer* gekapselt.

Außerdem brauchen wir drei Texturen, je eine für Farbe, Normale und Tiefeninformation. Eine Textur ist einfach ein Bildspeicher in OpenGL und wie für den Framebuffer gibt es eine Klasse *Texture*. Wir müssen für diese Texturen jedoch noch Speicher allokalieren und ihnen eine Größe und ein Format zuordnen, das machen wir mit `glTextureStorage2D`:

```
1 void resizeGBuffer() {
2     colorTexture = Texture<GL_TEXTURE_2D>();
3     glTextureStorage2D(colorTexture.handle, 1, GL_RGBA32F, resolution.x
4         , resolution.y);
5
6     normalTexture = Texture<GL_TEXTURE_2D>();
7     glTextureStorage2D(normalTexture.handle, 1, GL_RGBA16_SNORM,
8         resolution.x, resolution.y);
9
10    depthTexture = Texture<GL_TEXTURE_2D>();
```

```
9     glTextureStorage2D(depthTexture.handle, 1, GL_DEPTH32F_STENCIL8,  
10    resolution.x, resolution.y);  
11 }
```

Interessant sind hier vor allem die Formate: Die Farbe ist ein linearer 32-bit **float**-Vektor mit 4 Komponenten (**GL_RGBA32F**), die Normale ein ganzzahliger vorzeichenbehafteter 16-bit Vektor mit 4 Komponenten, der normalisiert ist, d.h. die höchste Zahl kodiert die Fließkomma 1 und die niedrigste Zahl die -1 (**GL_RGBA16_SNORM**), und neben der 32-bit float Tiefe speichern wir noch 8 Stencilbits, die man für verschiedene Zwecke benutzen kann (**GL_DEPTH32F_STENCIL8**). Die 1, die jeweils an zweiter Stelle steht, bezeichnet die Anzahl der Levels in der Textur, die man z.B. für Mipmaps benutzen kann, wir brauchen hier nur eine Ebene. Euch wird auffallen, dass die 4. Komponente der Normale und der Farbe unbenutzt bleibt, Grafikkarten rechnen jedoch lieber mit 4 Komponenten und diese 4. Komponente lässt sich theoretisch für Zusatzinformationen nutzen.

Wichtig zu erwähnen ist, dass so angelegte Texturen *immutable* sind, d.h. wir können weder das Format noch die Größe der Textur nach der Allokation ändern. Wir müssen also die Texturen jedes Mal neu anlegen, wenn sich die Fenstergröße ändert, das machen wir indem wir vorher den Konstruktor von **Texture** aufrufen. Der Typparameter in den eckigen Klammern ist der OpenGL-Typ der Textur, den wir anlegen wollen, in diesem Fall **GL_TEXTURE_2D**, für eine zweidimensionale Textur. Alternativ könnten wir z.B. auch ein- oder dreidimensionale Texturen oder auch sogenannte Cubemaps anlegen. Der Typ muss bei **glCreateTexture**-Aufrufen immer mitangegeben werden.

```
1  void resizeGBuffer() {  
2      colorTexture = Texture<GL_TEXTURE_2D>();  
3      glTextureStorage2D(colorTexture.handle, 1, GL_RGBA32F, resolution.x,  
4      resolution.y);  
5      glNamedFramebufferTexture(gBuffer.handle, GL_COLOR_ATTACHMENT0,  
6      colorTexture.handle, 0);  
7  
8      normalTexture = Texture<GL_TEXTURE_2D>();  
9      glTextureStorage2D(normalTexture.handle, 1, GL_RGBA16_SNORM,  
10     resolution.x, resolution.y);  
11     glNamedFramebufferTexture(gBuffer.handle, GL_COLOR_ATTACHMENT1,  
12     normalTexture.handle, 0);  
13  
14     depthTexture = Texture<GL_TEXTURE_2D>();  
15     glTextureStorage2D(depthTexture.handle, 1, GL_DEPTH32F_STENCIL8,  
16     resolution.x, resolution.y);  
17     glNamedFramebufferTexture(gBuffer.handle,  
18     GL_DEPTH_STENCIL_ATTACHMENT, depthTexture.handle, 0);  
19 }
```

Diese 3 Texturen müssen jetzt noch mit **glNamedFramebufferTexture** als Attachment an den Framebuffer gebunden werden. Der letzte Parameter bezieht sich hier wieder auf den Index des Levels der Textur, das wir binden wollen. Mit **glDrawBuffers** teilen wir OpenGL mit, dass wir diese auch

mit unserem Fragment-Shader beschreiben wollen:

```
1  resizeGBuffer();
2
3  gBuffer.bind();
4  std::array<GLenum, 2> drawBuffers = {GL_COLOR_ATTACHMENT0,
    GL_COLOR_ATTACHMENT1};
5  glDrawBuffers(drawBuffers.size(), drawBuffers.data());
```

Der Tiefenpuffer ist Teil der statischen Pipeline und wird von OpenGL beschrieben.

In unserer Rendermethode müssen wir den Framebuffer nun binden um ihn zu bemalen:

```
1  ////////////////////////////////////////////////// Geometry pass ///////////////////////////////////
2  gBuffer.bind();
3  glEnable(GL_DEPTH_TEST);
4  glDepthMask(GL_TRUE);
5  glClear(GL_DEPTH_BUFFER_BIT | GL_COLOR_BUFFER_BIT);
6
7  meshShader.set("uWorldToClip", worldToClip);
8  meshShader.set("uLocalToWorld", localToWorld);
9  meshShader.set("uGridSize", static_cast<int>(ceil(sqrt(bunnyCount))));
10 meshShader.bind();
11 mesh.draw(bunnyCount);
```

Es gibt aber noch eine weitere Neuerung: Statt nur einen Hasen zu malen, malen wir direkt alle Hasen auf einmal, indem wir der `draw` Methode eine Anzahl als Parameter übergeben:

```
1  void Mesh::draw(GLuint instances) {
2      vao.bind();
3      glDrawElementsInstanced(GL_TRIANGLES, numIndices, GL_UNSIGNED_INT,
        0, instances);
4  }
```

Das ist schneller da es die Kommunikation zwischen Grafikkarte und CPU reduziert, die sonst relativ teuer ist. Diese Technik heißt *Instancing*, das Problem ist jedoch, dass die Uniforms jetzt für jedes Objekt identisch sind, weshalb wir gerade den gleichen Hasen an der gleichen Stelle einfach mehrmals zeichnen. Um die einzelnen Instanzen zu unterscheiden stellt uns OpenGL im Vertex-Shader die Variable `int gl_InstanceID` zur Verfügung. Wir können `gl_InstanceID` dann benutzen um Objekteigenschaften prozedural zu generieren oder z.B. aus Uniform-Arrays auszulesen.

Aufgabe 1 (2 Punkte) Benutzt `gl_InstanceID`, um in `projection.vert` den einzelnen Instanzen verschiedene Positionen (`offset`) und Farben (`color`) zu geben. Ihr könnt die Hasen dafür in einem Gitter anordnen, `uGridSize` ist die gerundete Wurzel aus der Hasenanzahl und damit die Kantenlänge eines quadratischen Gitters, Ihr könnt die Hasen jedoch auch auf andere Arten anordnen.

Der Fragment-Shader (`geometry.frag`) bleibt beim Deferred-Rendering relativ minimal: Wir schrei-

ben einfach die interpolierten Vertex-Attribute in die dafür vorgesehen Texturen unseres Framebuffers:

```
1 #version 330 core
2
3 in vec3 interpNormal;
4 in vec3 interpPosition;
5 in vec3 color;
6
7 layout (location = 0) out vec4 gColor;
8 layout (location = 1) out vec4 gNormal;
9
10 void main() {
11     gColor.xyz = (mod(floor(interpPosition.y * 15.0), 2.0) * 0.5 + 0.5)
12                 * color;
13     gNormal.xyz = normalize(interpNormal);
14 }
```

Hier generieren wir zusätzlich ein prozedurales Streifenmuster, die Farbe wird jedoch meistens mittels Texturkoordinaten aus einer objektspezifischen Farbtextur gelesen.

Nach diesem Rendering-Pass liegen in unseren 3 Texturen die Farb-, Normalen- und Tiefeninformationen unserer Szene. Im Composing-Schritt binden wir diese 3 Texturen zum Lesen an unseren Shader und zeichnen damit ein Dreieck, das den gesamten Bildschirm abdeckt, in unser Fenster:

```
1 /////////////////////////////////////////////////// Lighting pass ///////////////////////////////////
2 Framebuffer::bindDefault();
3 glDisable(GL_DEPTH_TEST);
4 glDepthMask(GL_FALSE);
5
6 glBindTextureUnit(0, colorTexture.handle);
7 glBindTextureUnit(1, normalTexture.handle);
8 glBindTextureUnit(2, depthTexture.handle);
9 deferredCompose.set("uClipToWorld", inverse(worldToClip));
10 deferredCompose.set("uResolution", App::resolution);
11 deferredCompose.set("uLightPositions", uLightPositions);
12 deferredCompose.set("uLightColors", uLightColors);
13 deferredCompose.bind();
14 fullscreenTriangle.draw();
```

Mit `Framebuffer::bindDefault()` binden wir zuerst unseren Window-Framebuffer (Index 0) als aktiven Framebuffer. Die 3 Framebuffer Texturen binden wir jetzt jeweils an die drei Textureinheiten 0, 1, und 2. Eine **Textureinheit** ist dabei eine Hardwarekomponente der GPU, die eine Textur filtert und ausliest.

Hinweis: Da wir die Textur-Einheiten nie andersweitig benötigen, können wir die Texturen auch direkt nach der Erstellung in `resizeGBuffer` binden.

Der Vertex-Shader im Composing-Schritt bringt unser Dreieck einfach auf den Bildschirm und berechnet

uns eine Texturkoordinate $\in [0, 1]^2$, mit der wir aus unseren Framebuffer-Texturen lesen können:

```
1 #version 330 core
2
3 layout (location = 0) in vec3 position;
4
5 out vec2 texCoord;
6
7 void main() {
8     gl_Position = vec4(position, 1.0);
9     texCoord = position.xy * 0.5 + 0.5;
10 }
```

Im Fragment-Shader des Composing-Schritts (`compose.frag`) findet jetzt die eigentliche Arbeit statt:

```
1 #version 330 core
2
3 in vec2 texCoord;
4
5 uniform sampler2D tColor;
6 uniform sampler2D tNormal;
7 uniform sampler2D tDepth;
8
9 uniform mat4 uClipToWorld = mat4(1.0);
10 uniform vec2 uResolution = vec2(800, 600);
11 uniform float uBrightness = 1.0;
12
13 uniform int uLightCount;
14 uniform vec3 uLightPositions[256];
15 uniform vec3 uLightColors[256];
16
17 out vec3 fragColor;
18
19 void main() {
20     vec3 color = texture(tColor, texCoord).rgb;
21     vec3 normal = texture(tNormal, texCoord).xyz;
22     float depth = texture(tDepth, texCoord).r;
23     // TODO: Use texCoord or gl_FragCoord and depth to calculate the
24     //       fragment position in clip space
25     vec4 clipPos = vec4(0.0, 0.0, 0.0, 1.0);
26     // TODO: Obtain the world space position of the fragment
27     vec4 worldPosition = vec4(0.0, 0.0, 0.0, 1.0);
28     // Note: Divide worldPosition by w
29
30     vec3 lighting = vec3(0.0);
31     if (depth < 1.0) {
32         for (int i = 0; i < uLightCount; i++) {
33             vec3 lightPos = uLightPositions[i];
34             vec3 lightColor = uLightColors[i];
```

```

35         // Calculate lighting
36         vec3 lightDir = lightPos - worldPosition.xyz;
37         float lightDist = length(lightDir);
38         lightDir /= lightDist; // Normalize light direction
39         lightColor /= lightDist * lightDist; // Light falloff
40         lighting += max(dot(normal, lightDir), 0.0) * lightColor;
41     }
42     lighting *= uBrightness;
43     fragColor = lighting * color;
44 } else {
45     fragColor = vec3(0.0);
46 }
47 }

```

Wir lesen `color`, `normal` und `depth` mit der `texture-Funktion` aus unseren drei Texturen aus. Dafür müssen wir OpenGL über Uniforms noch mitteilen, in welchen Textureinheiten die einzelnen Texturen liegen:

```

1 deferredCompose.bindTextureUnit("tColor", 0);
2 deferredCompose.bindTextureUnit("tNormal", 1);
3 deferredCompose.bindTextureUnit("tDepth", 2);

```

`tColor`, `tNormal` und `tDepth` sind `sampler2D`, ein Sampler ist einfach ein Verweis auf eine Textureinheit mit einem Typ, der beschreibt was für eine Textur die Textureinheit hält.

Für Pixel, die im Vordergrund liegen (`depth < 1.0`) laufen wir jetzt über alle Lichtquellen und summieren die Beleuchtungsterme auf. Da wir Punktlichtquellen haben, brauchen wir die World-Space-Koordinate des Fragments, um die Helligkeit zu berechnen, die invers quadratisch abhängig vom Abstand der Lichtquelle ist. Die World-Space-Koordinate lässt sich jedoch einfach aus der Fragment Tiefe und Bildschirmposition ableiten.

Aufgabe 2 (3 Punkte) Benutzt `depth` und `texCoord` oder `gl_FragCoord` und `uResolution` um die Fragment-Position in Clip-Space bzw. Normalized-Device-Coordinates zu ermitteln und transformiert diese dann in den World-Space um `worldPosition` zu berechnen.

Die Farben und Positionen der Punktlichtquellen werden auf der CPU generiert und als Uniform-Arrays an den Fragment-Shader übergeben:

```

1 vec3 xyzCircle(float r, float t) { return r * vec3(cos(t), 0.0f, sin(t)); }
2
3 void genLightPositions(std::vector<vec3>& lightPositions, float radius,
4     float t, size_t count) {
5     lightPositions.resize(count);
6     srand(42);
7     for (size_t i = 0; i < count; i++) {
8         vec4 rand = linearRand(vec4(-1.0f), vec4(1.0f));

```

```

8      lightPositions[i] = xzCircle(sqrt(abs(rand.x)), t * rand.y) +
      xzCircle(sqrt(abs(rand.z)), t * rand.w);
9      lightPositions[i] *= radius;
10     lightPositions[i].y = linearRand(0.0f, 1.0f);
11 }
12 }
13
14 void genLightColors(std::vector<vec3>& lightColors, size_t count) {
15     lightColors.reserve(count);
16     for (size_t i = lightColors.size(); i < count; i++) {
17         lightColors.push_back(linearRand(vec3(0.0f), vec3(1.0f)));
18     }
19 }

```

Die Farben ziehen wir zufällig mit `glm::linearRand`, für die animierten Positionen benutzen wir ein sogenanntes **Double Pendulum**, d.h. wir addieren zwei Kreisbahnen mit zufälligen Radii und Winkelgeschwindigkeiten. Dadurch entsteht eine chaotische aber glatte Bewegung. Die Wurzel vor der Zufallsvariable, die den Radius bestimmt, sorgt dafür, dass sich die Lichtquellen etwas gleichmäßiger verteilen.

Theoretischer Aufgabenteil (5 Punkte)

Aufgabe 1

- **(a)** Gegeben sind zwei Punkte $\mathbf{p}_1, \mathbf{p}_2 \in P(\mathbb{R}^4)$, repräsentiert durch $\mathbf{h}_1 = (14, 3, 4, 2)^T$, $\mathbf{h}_2 = (12, 0, 3, 3)^T \in \mathbb{R}^4$. Gib die Projektionen von $\mathbf{p}_1$ und $\mathbf{p}_2$ in den $\mathbb{R}^3$ an. (1 Punkt)
- **(b)** Für welche Punkte im $P(\mathbb{R}^4)$ ist diese Projektion nicht möglich? Als was können diese Punkte interpretiert werden? (1 Punkt)

Aufgabe 2

Sei $P : \mathbb{P}(\mathbb{R}^2) \rightarrow \mathbb{P}(\mathbb{R}^2)$ eine perspektivische Projektion entlang der negativen y-Achse auf die x-Achse mit Augpunkt $\mathbb{R}^3 \ni A = (0, -y_0)^T$. Diese Projektion kann in eine perspektivische Transformation gefolgt von einer parallelen Projektion entlang der negativen y-Achse zerlegt werden.

Gib die entsprechenden Transformationsmatrizen für die perspektivische Projektion, die korrespondierende perspektivische Transformation und die darauf folgende parallele Projektion in folgender Form an (3 Punkte):

$$\begin{pmatrix} ? & ? & ? \\ ? & ? & ? \\ ? & ? & ? \end{pmatrix} = \begin{pmatrix} ? & ? & ? \\ ? & ? & ? \\ ? & ? & ? \end{pmatrix} \cdot \begin{pmatrix} ? & ? & ? \\ ? & ? & ? \\ ? & ? & ? \end{pmatrix}.$$